

Nume: Popa Petre Cosmin
Specializare: Mate-Info An 3

SecureDocs

Proof of Concept – Aplicație de stocare documente

Capitolul 1: Diagrama de Arhitectură

Aplicația *SecureDocs* este proiectată pe baza unei arhitecturi moderne, distribuite, care separă clar responsabilitățile fiecărei componente. Scopul acestei arhitecturi este de a oferi scalabilitate, securitate și mentenabilitate, respectând bunele practici DevOps.

Fluxul de date este următorul:

User → Load Balancer → Application Server → Storage

Utilizatorul accesează aplicația prin intermediul unui Load Balancer (Nginx), care reprezintă singurul punct de acces public. Load Balancer-ul distribuie cererile către unul dintre serverele de aplicație disponibile (App Server 1 sau App Server 2), asigurând o distribuție uniformă a traficului.

Serverele de aplicație sunt plasate într-o zonă privată a rețelei și nu sunt accesibile direct din exterior. Acestea sunt responsabile de logica aplicației și de interacțiunea cu sistemul de stocare. Stocarea documentelor este realizată printr-un serviciu de tip object storage (MinIO), aflat de asemenea în zona privată a rețelei.

Arhitectura este împărțită logic în două zone:

- **Zona Publică**, care include utilizatorul și Load Balancer-ul
- **Zona Privată**, care include serverele de aplicație și serviciul de stocare

Această separare reduce suprafața de atac și crește nivelul de securitate al sistemului.

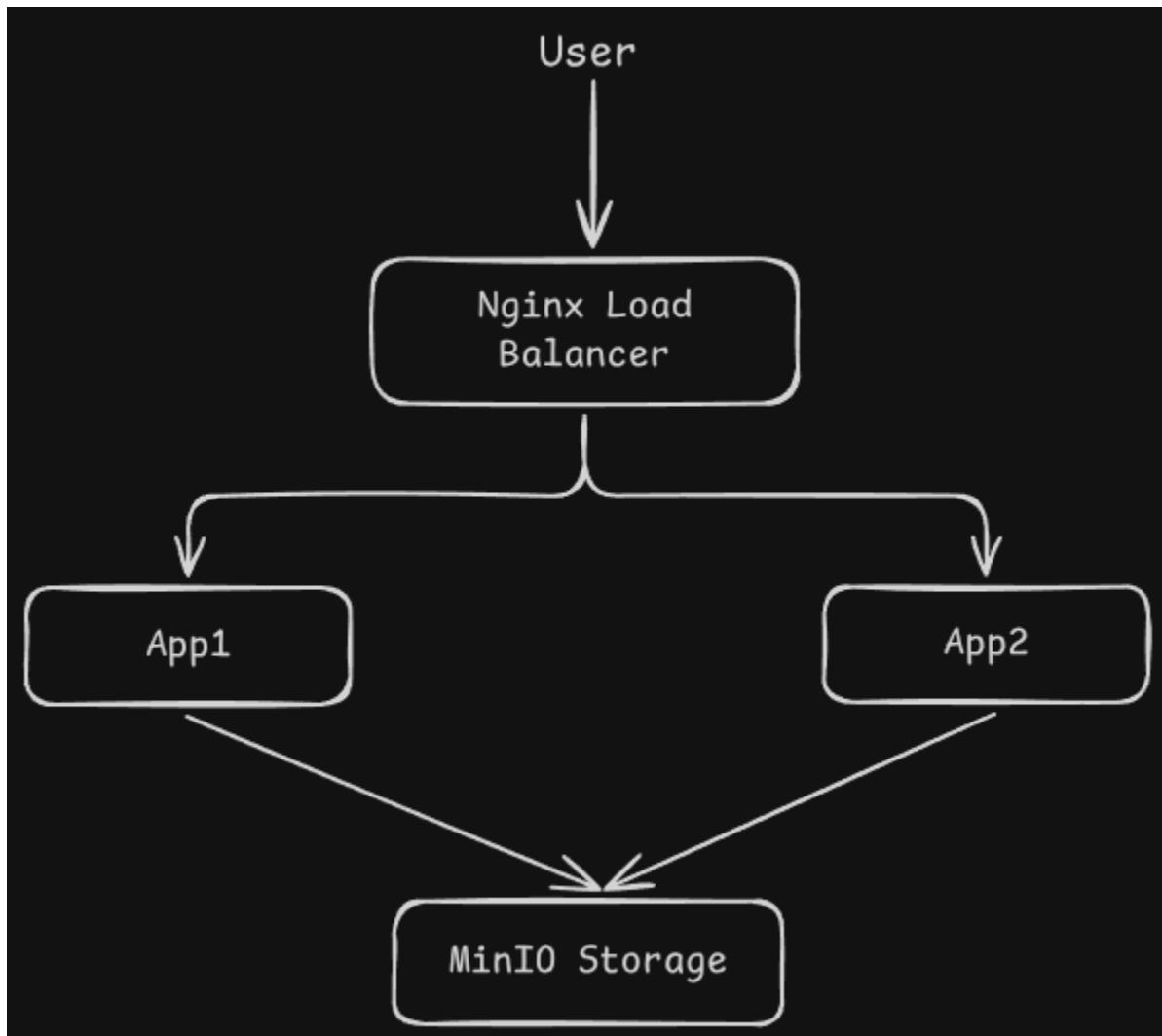


Figura 1 – Arhitectura aplicației SecureDocs

Capitolul 2: Stack-ul Tehnologic și Argumentarea Alegerilor

În dezvoltarea acestui Proof of Concept au fost alese tehnologii moderne, utilizate frecvent în mediile enterprise.

Docker

Docker a fost ales pentru containerizarea aplicației deoarece permite izolarea componentelor, rularea consistentă pe orice sistem și o mentenanță ușoară. Utilizarea containerelor permite replicarea rapidă a serverelor de aplicație și facilitează scalarea orizontală.

Docker Network

O rețea Docker dedicată a fost utilizată pentru a izola comunicația internă dintre containere. Astfel, doar Load Balancer-ul este expus public, iar celelalte servicii sunt accesibile exclusiv în interiorul infrastructurii.

Nginx

Nginx a fost ales ca Reverse Proxy și Load Balancer datorită performanței ridicate, consumului redus de resurse și suportului nativ pentru load balancing. Comparativ cu Apache, Nginx este mai eficient în gestionarea unui număr mare de conexiuni simultane.

MinIO

MinIO a fost ales ca soluție de stocare deoarece este un sistem de tip object storage compatibil cu Amazon S3, foarte utilizat în mediile cloud. Oferă performanță ridicată, persistență a datelor și posibilitatea extinderii ulterioare.

Ubuntu 22.04 LTS

Sistemul de operare ales este Ubuntu 22.04 LTS, datorită stabilității, suportului pe termen lung și compatibilității excelente cu Docker și ecosistemul DevOps.

Capitolul 3: Soluția de Stocare

Stocarea documentelor este realizată utilizând MinIO, un serviciu de tip object storage. Documentele sunt organizate într-un bucket dedicat, numit documents, care reprezintă spațiul logic de stocare pentru fișierele utilizatorilor.

Această abordare oferă multiple avantaje:

- separarea clară între aplicație și date
- persistența documentelor indiferent de ciclul de viață al containerelor
- posibilitatea de scalare independentă a stocării

Persistența datelor este asigurată prin utilizarea volumelor Docker, astfel încât fișierele să nu fie pierdute în cazul opririi sau recreării containerului MinIO.

Într-un scenariu real de producție, redundanța și backup-ul ar putea fi realizate prin:

- replicarea bucket-urilor
- sincronizarea cu un storage extern
- snapshot-uri periodice

Arhitectura aleasă permite integrarea acestor mecanisme fără modificări majore.

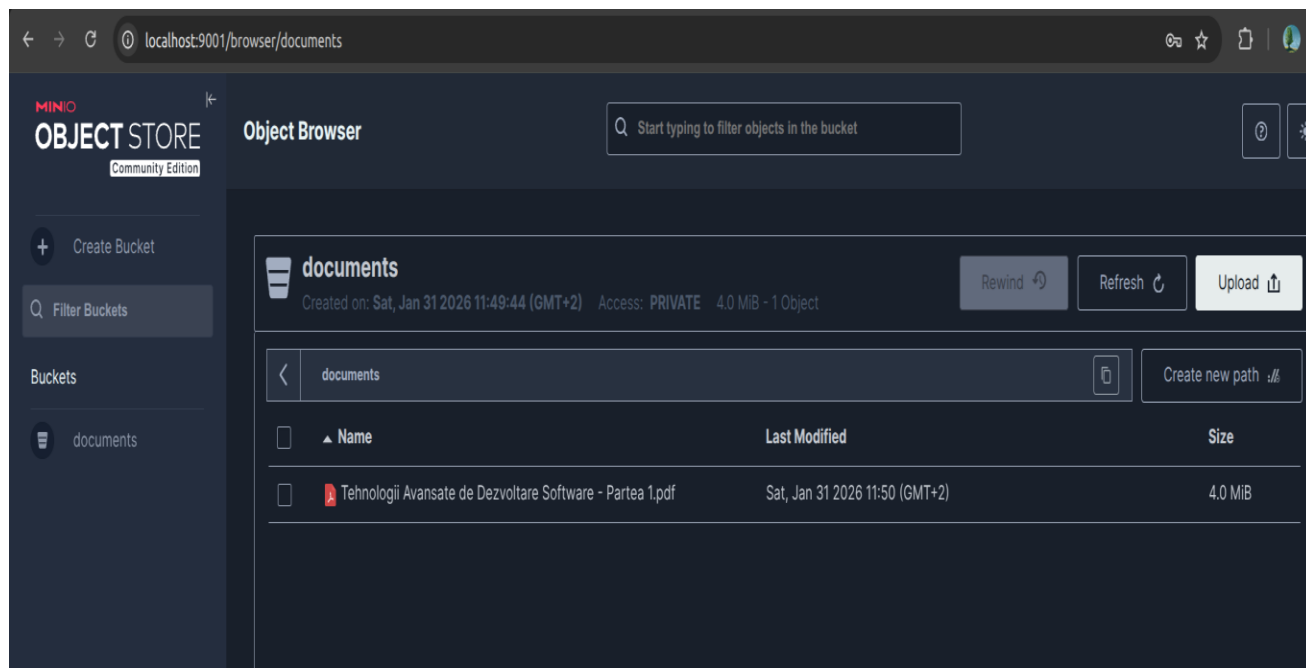


Figura 2 – Bucket-ul documents în MinIO + exemplu document încărcat în sistem

Capitolul 4: Scenarii de Mentenanță

Scalare

În cazul în care numărul de utilizatori se dublează, componenta care trebuie multiplicată este stratul de aplicație. Datorită utilizării containerelor Docker, pot fi pornite rapid noi instanțe de servere de aplicație, iar Load Balancer-ul le poate include automat în distribuția traficului.

Update (Zero Downtime Deployment)

Actualizarea aplicației fără întreruperea serviciului se poate realiza printr-o strategie de tip rolling update. O nouă versiune a aplicației este pornită în paralel cu cea existentă, iar Load Balancer-ul redirecționează treptat traficul către noile instanțe. După finalizarea tranziției, instanțele vechi sunt oprite.

Această abordare minimizează impactul asupra utilizatorilor și este larg utilizată în sistemele moderne.

Capitolul 5: Monitorizare și Logare

În cadrul acestui Proof of Concept nu a fost implementată o soluție completă de monitorizare, însă într-un scenariu real ar putea fi utilizată combinația Prometheus + Grafana.

Monitorizarea este asigurată prin:

- colectarea metricilor de stare ale containerelor
- verificarea disponibilității serviciilor
- alerte configurate pentru situații critice

Pentru logare, se poate utiliza o soluție centralizată precum **ELK Stack (Elasticsearch, Logstash, Kibana)**. Logurile aplicației sunt colectate automat din containere și centralizate într-un singur punct, facilitând analiza și depanarea.

Funcționalitatea sistemului de logare este asigurată prin:

- rularea serviciilor de logare ca servicii separate
- persistența datelor
- monitorizarea stării acestora

Demonstrație Practică

```
→ ~ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4f1198144c95   minio/minio   "/usr/bin/docker-ent..." 13 days ago   Up 5 seconds  0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   minio
81d52c4c9bfe   nginx         "/docker-entrypoint..." 2 weeks ago   Up 6 seconds  0.0.0.0:80->80/tcp, [::]:80->80/tcp   nginx-lb
68860532c351   nginx         "/docker-entrypoint..." 2 weeks ago   Up 6 seconds  80/tcp                               app2
190390a435f0   nginx         "/docker-entrypoint..." 2 weeks ago   Up 6 seconds  80/tcp                               app1
→ ~
```

Figura 3 – Containerele aplicației rulând în Docker

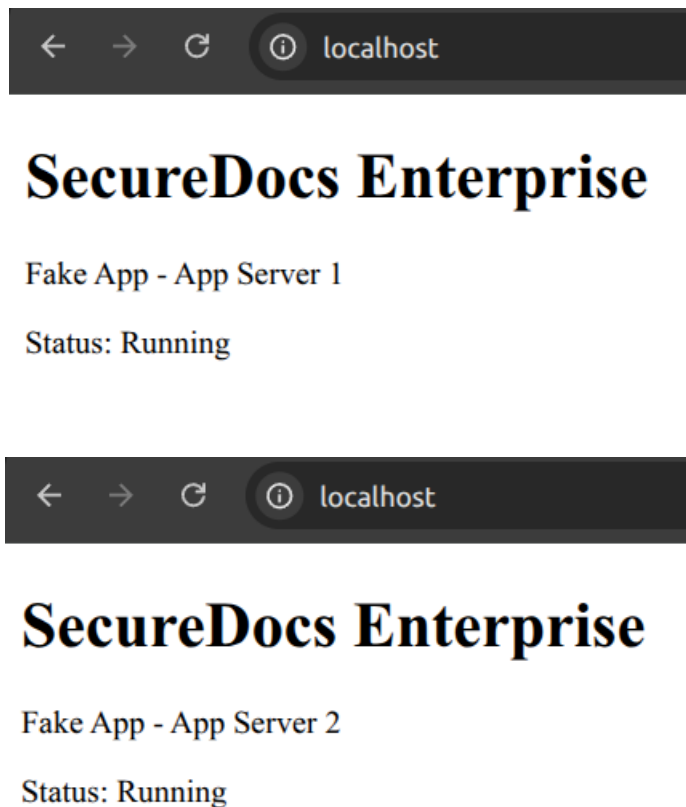


Figura 4 – Aplicația accesată prin Load Balancer

Concluzie

Proiectul *SecureDocs* reprezintă un Proof of Concept care demonstrează implementarea unei arhitecturi moderne, sigure și scalabile pentru gestionarea documentelor. Prin utilizarea containerelor, a load balancing-ului și a unui sistem de stocare dedicat, soluția respectă bunele practici DevOps și oferă o bază solidă pentru extindere ulterioară într-un mediu de producție.